

SYSTEM REQUIREMENTS SPECIFICATION (SRS)

Project: Clinic Scheduling Management System (Clinic API)

1. Introduction

This System Requirements Specification (SRS) document provides a detailed and comprehensive technical and business overview of the Clinic API project. The system is designed as a specialized Backend API, optimized for data processing and ready for integration with modern Frontend applications (such as React, Vue, or mobile apps).

The core objectives of the system include automating the doctor's schedule management process, simplifying the patient's appointment booking operations, minimizing the risk of data conflicts (double-booking), and enhancing healthcare experience through automated reminder mechanisms.

2. Scope & Actors

The system focuses on solving the scheduling interaction between two core actors: * **Doctor:** Actively sets up and updates their daily availability (Schedules). Manages the list of patients who have booked appointments under their personal management. * **Patient:** Searches for doctors by specialty, finds and registers for available slots in the system, and tracks personal appointment status.

3. Architecture & Technology Stack

To ensure high load capacity, flexible scalability, and fast response times, the system adheres to the following architectural standards: * **Core Framework:** Ruby on Rails configured in API-only mode (`--api`), removing unnecessary interface middlewares to optimize memory usage and request processing speed. * **Code Organization:** Strictly follows the Single Responsibility Principle. Clear separation between routing logic (Skinny Controllers), complex business logic (Service Objects), and optimized database interactions (Active Record

Models). * **Database Management System:** Relational Database (PostgreSQL for production / SQLite for development).

4. Database Schema Design

The system consists of 3 core database tables forming the main business logic:

4.1. Users Table (Account Management & Authorization)

Field Name	Data Type	Constraints / Description
<code>id</code>	Bigint	Primary Key, Auto-increment
<code>email</code>	String	Unique, Presence, Valid Email Format
<code>password_digest</code>	String	Presence (Encrypted password using Bcrypt)
<code>name</code>	String	Presence (User's full name)
<code>role</code>	String / Enum	Values in: <code>[:doctor, :patient]</code>

4.2. Schedules Table (Doctor Working Hours)

Field Name	Data Type	Constraints / Description
<code>id</code>	Bigint	Primary Key, Auto-increment
<code>user_id</code>	Bigint / Foreign Key	Links to <code>Users</code> table (Accepts only Users with <code>role = doctor</code>)
<code>date</code>	Date	Presence (Appointment Date)
<code>start_time</code>	Time	Presence (Shift Start Time)
<code>end_time</code>	Time	Presence (Shift End Time)

4.3. Appointments Table (Clinical Appointments)

Field Name	Data Type	Constraints / Description
<code>id</code>	Bigint	Primary Key, Auto-increment
<code>patient_id</code>	Bigint / Foreign Key	Links to <code>Users</code> table (Accepts only Users with <code>role = patient</code>)
<code>schedule_id</code>	Bigint / Foreign Key	Unique (Anti Double-Booking), Links to <code>Schedules</code> table
<code>status</code>	String / Enum	Values in: <code>[:pending, :confirmed, :cancelled]</code>

5. Core Business Logic

5.1. Authentication & Authorization

- **JWT Mechanism:** All APIs (except signup/login) are strictly secured. Upon successful login, the system generates a JWT token containing `user_id` and expiration time. The Frontend must attach this token to the HTTP Header (`Authorization: Bearer <token>`).
- **Data Isolation:** All personal data queries must scope through the authenticated user via Controller variables (e.g., `current_user.appointments`). This strictly prevents unauthorized ID tampering in URLs.

5.2. Anti Double-Booking Logic

This is the most critical rule of the system. A doctor's availability slot (Schedule) at any given time can only be assigned to a single valid Appointment. This logic is protected at two layers:

- * **Database Layer:** A Unique Index is defined on the `schedule_id` column in the `appointments` table. This completely prevents race conditions when two patients attempt to book the exact same slot at the exact same millisecond.
- * **Active Record Validation Layer:** Validates the existence of an appointment prior to saving using strict model-level conditional checks.

5.3. Time Validations

- Doctors cannot create a working slot (Schedule) in the past.
- The end time (`end_time`) of a shift must be after its start time (`start_time`).
- Patients are not allowed to book appointments for shifts that have already passed.

5.4. Background Jobs

The system integrates Sidekiq (or Active Job defaults combined with Redis) to offload time-consuming tasks from the main request-response cycle, keeping API response times under 50ms.

- * **Automated Reminder Email:** When an appointment status changes to `:confirmed` , a background worker is scheduled to automatically send a reminder email to the patient exactly 24 hours before the appointment time.

6. Error Handling & Performance Optimization

6.1. Anti N+1 Query Strategy

When querying lists of appointments or doctors with their schedules, the system applies Eager Loading by chaining `.includes` in the Service Object layer. Related data is batched and retrieved in a maximum of 2 SQL queries instead of repeating query execution for every record (N+1).

6.2. Global Rescue Network

Unexpected runtime exceptions (e.g., Record Not Found or parameter syntax errors) are caught centrally at `ApplicationController` using the `rescue_from` declaration.

```
# Error handling simulation in ApplicationController
class ApplicationController < ActionController::API
  rescue_from ActiveRecord::RecordNotFound, with: :render_not_found

  private

  def render_not_found(exception)
    render json: { message: "Requested record not found" }, status: :not_found
  end
end
```